

```

In [5]: ClearAll["Global`*"];
$HistoryLength=0;

Print["====="];
Print["S124-T2 STANDALONE"];
Print["TRANSFORMER PERCEPTION -> FROZEN TCCT-Q INTERFACE GATE"];
Print["====="];
Print["WolframVersion=", $Version];
Print["Date=",DateString[]];

T2Stop[msg_] := (Print["FATAL: ", msg]; Abort[]);

r3mDepth=6;
r3mAnchors={1,2,3};
r3mHub=10;
r3mWLRounds=4;
r3mSupports={2,4,6};
r3mEffects={0.,0.05};
r3mTopMins={0.30,0.35};
r3mMinChildStates=2;
r3mMinGroupObs=80;
r3mMinValGain=0.001;
r3mPenalty=0.00025;
r3mMaxSplits=12;
r3mMaxBlocks=24;
r3mKappa=8.;
r3mZero=ConstantArray[0,{24,4}];

t2CoreSeed=1134001;
t2WorldSeed=6240101;
t2BankAB=15000;
t2BankC=6000;

t2PerceptionTrainPerClass=300;
t2PerceptionValPerClass=100;

t2SensorDim=12;
t2PosDim=8;
t2SensorScale=2.;
t2SensorNoise=0.25;
t2SensorDrift=0.10;
t2SensorGainJitter=0.10;

t2DModel=64;
t2Heads=4;
t2Layers=2;
t2FF=192;
t2Dropout=0.10;

t2LearningRate=0.0003;
t2BatchSize=64;
t2MaxRounds=20;
t2Patience=5;
t2TargetDevice="CPU";

```

```

t2PrototypeSeed=6241001;
t2PerceptionTrainSeed=6241002;
t2PerceptionValSeed=6241003;
t2PerceptionNetSeed=6241004;
t2SensorCSeed=6241005;
t2RandomControlSeed=6241006;

If[OddQ[t2PosDim],T2Stop["t2PosDim must be even"]];
If[Mod[t2DModel,t2Heads]!=0,T2Stop["dModel must be divisible by heads"]];

t2HeadDim=Quotient[t2DModel,t2Heads];

t2OutputDirectory=FileNameJoin[{Directory[],"S124_T2_Output"}];
If[!DirectoryQ[t2OutputDirectory],CreateDirectory[t2OutputDirectory]];

R3MFiniteQ[x_]:=NumericQ[x]&&FreeQ[x,Indeterminate|ComplexInfinity|_DirectedInfinity]
R3MUE[a_,b_]:=UndirectedEdge@@Sort[{a,b}];
R3MSeqKey[s_List]:=ToString[s,InputForm];
R3MCat[p_List]:=RandomChoice[N[p]->Range[Length[p]]];

R3MInitial[]:=<|
"Active"->Union[R3MUE[#,r3mHub]&/@r3mAnchors],
"Frozen"->{},
"Next"->100
|>;

R3MNeighbors[active_List,v_]:=
DeleteDuplicates@Join[
Cases[active,UndirectedEdge[a_,b_]/;a===v:>b],
Cases[active,UndirectedEdge[a_,b_]/;b===v:>a]
];

R3MCandidates[active_List,required_List]:=
Module[{verts,ns,cands={},x,y,z,e1,e2},
verts=DeleteDuplicates[Flatten[List@@@active]];
Do[
y=verts[[k]];
ns=Sort[R3MNeighbors[active,y]];
Do[
x=ns[[i]];
z=ns[[j]];
e1=R3MUE[x,y];
e2=R3MUE[y,z];
If[
Intersection[{e1,e2},required]!={},
AppendTo[cands,{x,y,z,e1,e2}]
],
{i,1,Max[0,Length[ns]-1]},
{j,i+1,Length[ns]}
],
{k,1,Length[verts]}
];
cands
];

```

```

R3MInject[state_Association,event_Integer,seed_Integer,step_Integer]:=
Module[
{active,frozen,next,anchor,fresh,w,inj,cands,c,x,y,z,e1,e2,newActive,newFrozen},
active=state["Active"];
frozen=state["Frozen"];
next=state["Next"];
anchor=r3mAnchors[[event]];
fresh=next;
w=next+1;
inj={R3MUE[anchor,fresh],R3MUE[fresh,r3mHub]};
active=Union[Join[active,inj]];
cands=R3MCandidates[active,inj];
If[
cands==={},
Return[
<|
"Active"->DeleteCases[active,e_/_;MemberQ[inj,e]],
"Frozen"->frozen,
"Next"->next+2
|>
]
];
c=BlockRandom[
SeedRandom[seed+100003*step+7919*event];
RandomChoice[cands]
];
{x,y,z,e1,e2}=c;
newActive=DeleteCases[active,e_/_;MemberQ[{e1,e2},e]];
newActive=Union[
Join[
newActive,
{
R3MUE[x,z],
R3MUE[x,w],
R3MUE[w,z]
}
]
];
newActive=DeleteCases[newActive,e_/_;MemberQ[inj,e]];
newFrozen=Union[
Join[
frozen,
{
DirectedEdge[x,y],
DirectedEdge[y,z]
}
]
];
<|
"Active"->newActive,
"Frozen"->newFrozen,
"Next"->next+2
|>
];

R3MStateKey[state_Association]:=

```

```

Module[{active,frozen,verts,col,new,na,no,ni,others},
active=state["Active"];
frozen=state["Frozen"];
verts=Union[
r3mAnchors,
{r3mHub},
Flatten[List@@@active],
If[frozen=={},{},Flatten[List@@@frozen]]
];
col=Association@Table[
v->Which[
v===r3mAnchors[[1]],"A1",
v===r3mAnchors[[2]],"A2",
v===r3mAnchors[[3]],"A3",
v===r3mHub,"H",
True,"X"
],
{v,verts}
];
Do[
new=Association@Table[
na=Sort@Join[
Cases[active,UndirectedEdge[a_,b_]/;a===v:>col[b]],
Cases[active,UndirectedEdge[a_,b_]/;b===v:>col[a]]
];
no=Sort[
Cases[
frozen,
DirectedEdge[a_,b_]/;a===v:>col[b]
]
];
ni=Sort[
Cases[
frozen,
DirectedEdge[a_,b_]/;b===v:>col[a]
]
];
v->Hash[
{col[v],na,no,ni},
"SHA256",
"HexString"
],
{v,verts}
];
col=new,
{r3mWLRounds}
];
others=Sort[
Lookup[
col,
Complement[
verts,
Join[r3mAnchors,{r3mHub}]
]
]
];
];

```

```

Hash[
{
Lookup[col,r3mAnchors],
col[r3mHub],
others,
Length[active],
Length[frozen]
},
"SHA256",
"HexString"
]
];

R3MBuildCore[coreSeed_Integer]:=
Module[{rows,seq,state,seqKeys,keys},
rows={<|"Seq"->{},"State"->R3MInitial[]|>};
Do[
rows=Flatten[
Table[
Table[
seq=Append[row["Seq"],ev];
state=R3MInject[
row["State"],
ev,
coreSeed,
d
];
<|"Seq"->seq,"State"->state|>,
{ev,1,3}
],
{row,rows}
],
1
];
Print["CoreDepth=",d," Histories=",Length[rows]],
{d,1,r3mDepth}
];
seqKeys=R3MSeqKey/@Lookup[rows,"Seq"];
keys=R3MStateKey/@Lookup[rows,"State"];
<|
"SeqToState"->AssociationThread[seqKeys,keys],
"SeqKeys"->seqKeys,
"KeyList"->keys,
"States"->DeleteDuplicates[keys],
"UniqueStates"->Length[DeleteDuplicates[keys]]
|>
];

r3oSeqs=
Join[#{1,2}&/@DeleteDuplicates[
Permutations[{1,2,3,3}]
];

r3oSeqKeys=R3MSeqKey/@r3oSeqs;

If[Length[r3oSeqs]!=12,T2Stop["expected exactly 12 S3Q histories"]];

```

```

If[
!And@@(
(
Sort[#]==Sort[{1,1,2,2,3,3}]&&
Take[#, -2]=={1,2}
)&/@r3oSeqs
),
T2Stop["S3Q order invariant failed"]
];

Print[""];
Print["====="];
Print["S3Q HISTORY PRECHECK"];
Print["====="];
Print["HistoryCount=", Length[r3oSeqs]];
Print["AllEventBags={2,2,2}=True"];
Print["AllRecent2={1,2}=True"];

r3mFullAll=R3MBuildCore[t2CoreSeed];

R3ORestrictData[data_Association]:=
Module[{vals},
vals=data["SeqToState"][#]&/@r3oSeqKeys;
<|
"SeqToState"->AssociationThread[r3oSeqKeys, vals],
"States"->DeleteDuplicates[vals],
"UniqueStates"->Length[DeleteDuplicates[vals]]
|>
];

t2FullData=R3ORestrictData[r3mFullAll];

Print[""];
Print["RestrictedFullStructuralStates=", t2FullData["UniqueStates"]];

If[
t2FullData["UniqueStates"]!=12,
T2Stop[
"expected 12 Full TCCT structural states"
]
];

t2BagStates=
Length[
DeleteDuplicates[
Hash[
{
"BAG",
Count[#, 1],
Count[#, 2],
Count[#, 3]
},
"SHA256",
"HexString"
]&/@r3oSeqs
]
]

```

```

]
];

t2RecentStates=
Length[
DeleteDuplicates[
Hash[
{"REC2",Take[#, -2]},
"SHA256",
"HexString"
]&/@r3oSeqs
]
];

If[t2BagStates!=1,T2Stop["EventBag shortcut precheck failed"]];
If[t2RecentStates!=1,T2Stop["Recent2 shortcut precheck failed"]];

Print["EventBagStates=", t2BagStates];
Print["Recent2States=", t2RecentStates];
Print["TCCT CORE PRECHECK=PASS"];

R3OP[k_Integer]:=
0.08*ConstantArray[1.,4]+0.68*UnitVector[4,k];

R3OWorld[seed_Integer]:=
BlockRandom[
SeedRandom[seed];
Module[{perm,regs,seqReg,off1,off2,p1,p2},
perm=RandomSample[r3oSeqKeys];
regs=Flatten[
Table[
ConstantArray[r,4],
{r,1,3}
]
];
seqReg=AssociationThread[perm,regs];
off1=RandomInteger[{0,3}];
off2=RandomInteger[{0,3}];
p1=Table[
R3OP[
1+Mod[
r+o+2*a+off1-2,
4
]
],
{r,1,3},
{o,1,4},
{a,0,1}
];
p2=Table[
R3OP[
1+Mod[
2*r+o+a6+2*a7+off2-2,
4
]
],

```

```

{r,1,3},
{o,1,4},
{a6,0,1},
{a7,0,1}
];
<|
"SeqRegime"->seqReg,
"P1"->p1,
"P2"->p2
|>
]
];

R3OSim[seed_Integer,world_Association]:=
BlockRandom[
SeedRandom[seed];
Module[{seq,key,reg,o,a6,a7,y1,y2},
seq=RandomChoice[r3oSeqs];
key=R3MSeqKey[seq];
reg=world["SeqRegime"][key];
o=RandomInteger[{1,4}];
a6=RandomInteger[{0,1}];
a7=RandomInteger[{0,1}];
y1=R3MCat[
world["P1"][[reg,o,a6+1]]
];
y2=R3MCat[
world["P2"][[reg,o,a6+1,a7+1]]
];
<|
"SeqKey"->key,
"O6"->o,
"A6"->a6,
"A7"->a7,
"Y1"->y1,
"Y2"->y2
|>
]
];

R3MProbe1[o_Integer,a6_Integer]:=
1+2*(o-1)+a6;

R3MProbe2[o_Integer,a6_Integer,a7_Integer]:=
9+4*(o-1)+2*a6+a7;

R3MCountsRows[rows_List]:=
Module[{m=ConstantArray[0,{24,4}],p1,p2},
Do[
p1=R3MProbe1[
row["O6"],
row["A6"]
];
p2=R3MProbe2[
row["O6"],
row["A6"],

```



```

row["A7"]
];
m[[p1,row["Y1"]]]++;
m[[p2,row["Y2"]]]++;
{row,rows}
];
m
];

R3MStateCountsData[rows_List,data_Association]:=
Association@KeyValueMap[
Function[
{k,rs},
k->R3MCountsRows[rs]
],
GroupBy[
rows,
data["SeqToState"][#["SeqKey"]]&
]
];

R3MGet[c_Association,s_]:=
Lookup[c,s,r3mZero];

R3MPooled[c_Association,states_List]:=
Total[
R3MGet[c,#]&/@states
];

R3MObs[c_Association,states_List]:=
Total[
Flatten[
R3MPooled[c,states]
]
];

R3MAddCounts[a_Association,b_Association,states_List]:=
Association@Table[
s->(
R3MGet[a,s]+
R3MGet[b,s]
),
{s,states}
];

R3MPrepare[rawA_List,rawB_List,data_Association]:=
Module[{a,b,states},
states=data["States"];
a=R3MStateCountsData[rawA,data];
b=R3MStateCountsData[rawB,data];
<|
"States"->states,
"A"->a,
"B"->b,
"AB"->R3MAddCounts[
a,

```

```

b,
states
]
|>
];

R3MLabel[
v_,
support_Integer,
effect_?NumericQ,
topMin_?NumericQ
]:=
Module[{n,p,ord,top,runner},
If[
!VectorQ[v,NumericQ]||Length[v]!=4,
T2Stop["R3MLabel shape error"]
];
n=Total[v];
If[n<support,Return[0]];
p=N[v/n];
ord=Reverse[Ordering[p]];
top=ord[[1]];
runner=ord[[2]];
If[
p[[top]]>=topMin&&
p[[top]]-p[[runner]]>=effect,
top,
0
]
];

R3MMakeGroups[
states_List,
cA_Association,
cB_Association,
probe_Integer,
support_Integer,
effect_?NumericQ,
topMin_?NumericQ
]:=
Module[{lab,ga,keys,pos,groups},
lab=Association@Table[
s->R3MLabel[
R3MGet[cA,s][[probe]],
support,
effect,
topMin
],
{s,states}
];
ga=KeySort@GroupBy[
states,
lab[#]&
];
keys=Keys[ga];
pos=Select[keys,#>0&];

```

```

If[Length[pos]<2,Return[$Failed]];
If[
Min[Length/@Lookup[ga,pos]]<r3mMinChildStates,
Return[$Failed]
];
groups=Values[ga];
If[
Min[R3MObs[cA,#]&/@groups]<r3mMinGroupObs,
Return[$Failed]
];
If[
Min[R3MObs[cB,#]&/@groups]<r3mMinGroupObs,
Return[$Failed]
];
<|
"Groups"->groups,
"GroupLabels"->keys
|>
];

R3MLossBase[train_,test_]:=
Module[{loss=0.,n=0,prob,den},
Do[
den=Total[train[[p]]]+4.;
prob=(N[train[[p]]]+1.)/den;
loss-=N[test[[p]].Log[prob]];
n+=Total[test[[p]]],
{p,1,24}
];
{loss,n}
];

R3MLossShrink[train_,parent_,test_]:=
Module[{loss=0.,n=0,parentP,prob,den},
Do[
parentP=
(N[parent[[p]]]+1.)/
(Total[parent[[p]]]+4.);
den=
Total[train[[p]]]+
r3mKappa;
prob=
(
N[train[[p]]]+
r3mKappa*parentP
)/den;
loss-=N[test[[p]].Log[prob]];
n+=Total[test[[p]]],
{p,1,24}
];
{loss,n}
];

R3MSplitEval[
states_List,
groups_List,

```

```

cA_Association,
cB_Association
]:=
Module[
{pa,pb,parentLoss,splitLoss=0.,n=0,ca,cb,z,gain},
pa=R3MPooled[cA,states];
pb=R3MPooled[cB,states];
parentLoss=R3MLossBase[pa,pb];
Do[
ca=R3MPooled[cA,g];
cb=R3MPooled[cB,g];
z=R3MLossShrink[ca,pa,cb];
splitLoss+=z[[1]];
n+=z[[2]],
{g,groups}
];
If[
n<=0||n!=parentLoss[[2]],
Return[$Failed]
];
gain=
N[
parentLoss[[1]]/n-
splitLoss/n
];
If[
!R3MFiniteQ[gain],
T2Stop["nonfinite split gain"]
];
<|"ValGain"->gain|>
];

R3MBestSplit[
states_List,
cA_Association,
cB_Association
]:=
Module[
{best=$Failed,bestScore=-Infinity,g,ev,score},
Do[
g=R3MMakeGroups[
states,
cA,
cB,
probe,
support,
effect,
topMin
];
If[
UnsameQ[g,$Failed],
ev=R3MSplitEval[
states,
g["Groups"],
cA,
cB

```

```

];
If[
  UnsameQ[ev,$Failed],
  score=
  N[
    ev["ValGain"]-
    r3mPenalty*
    (Length[g["Groups"]]-1)
  ];
  If[
    score>bestScore,
    bestScore=score;
    best=Join[
      <|
        "Probe"->probe,
        "Support"->support,
        "Effect"->effect,
        "TopMin"->topMin
      |>,
      g,
      ev,
      <|"Score"->score|>
    ]
  ]
],
{probe,1,24},
{support,r3mSupports},
{effect,r3mEffects},
{topMin,r3mTopMins}
];
best
];

R3MDiscover[
  states_List,
  cA_Association,
  cB_Association
]:=
Module[
  {blocks={states},log={},step=0,cands,best,bi},
  While[
    step<r3mMaxSplits&&
    Length[blocks]<r3mMaxBlocks,
    cands=Cases[
      Table[
        If[
          Length[blocks[[i]]]<
            2*r3mMinChildStates,
          Nothing,
          With[
            {x=R3MBestSplit[
              blocks[[i]],
              cA,
              cB
            ]},

```

```

If[
  UnsameQ[x,$Failed],
  Join[
    <|
      "BlockIndex"->i,
      "ParentStates"->
        Length[blocks[[i]]]
    |>,
    x
  ],
  Nothing
]
]
],
{i,1,Length[blocks]}
],
_Association
];
If[cands==={},Break[]];
best=First[
  MaximalBy[
    cand,
    #["Score"]&
  ]
];
If[
  best["ValGain"]<r3mMinValGain||
  best["Score"]<=0.,
  Break[]
];
bi=best["BlockIndex"];
step++;
Print[
  "ACCEPT SPLIT=",step,
  " STATES=",best["ParentStates"],
  " PROBE=",best["Probe"],
  " GAIN=",N[best["ValGain"]]
];
AppendTo[
  log,
  KeyDrop[
    best,
    {"Groups"}
  ]
];
blocks=Join[
  Take[blocks,bi-1],
  best["Groups"],
  Drop[blocks,bi]
];
];
];
<|
  "Blocks"->blocks,
  "Log"->log,
  "Splits"->step
|>

```

```

];

R3MBlockMap[blocks_List]:=
Association@Flatten[
MapIndexed[
Thread[#1->First[#2]]&,
blocks
]
];

R3MProspective[
blocks_List,
cTrain_Association,
cTest_Association,
states_List
]:=
Module[
{map,blockTrain,globalTrain,baseLoss=0.,exactLoss=0.,qLoss=0.,n=0,test,z,bid},
map=R3MBlockMap[blocks];
globalTrain=R3MPooled[
cTrain,
states
];
blockTrain=AssociationThread[
Range[Length[blocks]],
R3MPooled[cTrain,#]&/@blocks
];
Do[
test=R3MGet[cTest,s];
z=R3MLossBase[
globalTrain,
test
];
baseLoss+=z[[1]];
z=R3MLossShrink[
R3MGet[cTrain,s],
globalTrain,
test
];
exactLoss+=z[[1]];
bid=map[s];
z=R3MLossShrink[
blockTrain[bid],
globalTrain,
test
];
qLoss+=z[[1]];
n+=Total[Flatten[test]],
{s,states}
];
If[n<=0,T2Stop["no prospective observations"]];
<|
"BaseNLL"->N[baseLoss/n],
"ExactStateNLL"->N[exactLoss/n],
"QuotientNLL"->N[qLoss/n],
"ExactGainVsBase"->N[(baseLoss-exactLoss)/n],

```

```

"QuotientGainVsBase"->N[(baseLoss-qLoss)/n],
"QuotientGainVsExact"->N[(exactLoss-qLoss)/n]
|>
];

R3MEvaluate[
model_Association,
rawC_List,
data_Association
]:=
Module[{cC,p},
cC=R3MStateCountsData[
rawC,
data
];
p=R3MProspective[
model["Blocks"],
model["TrainCounts"],
cC,
model["States"]
];
Join[
model,
p
]
];

Print[""];
Print["====="];
Print["PHASE 1: DISCOVERY A + VALIDATION B"];
Print["PROSPECTIVE C DOES NOT EXIST"];
Print["====="];

t2World=R30World[t2WorldSeed];

t2RawA=Table[
R30Sim[
t2WorldSeed+61000000+i,
t2World
],
{i,1,t2BankAB}
];

t2RawB=Table[
R30Sim[
t2WorldSeed+62000000+i,
t2World
],
{i,1,t2BankAB}
];

t2Prep=R3MPrepare[
t2RawA,
t2RawB,
t2FullData
];

```



```

Print["StructuralStates=", Length[t2Prep["States"]]];

If[
Length[t2Prep["States"]] != 12,
T2Stop["structural state count changed before quotient discovery"]
];

Print["Discovering predictive quotient..."];

t2Disc=R3MDiscover[
t2Prep["States"],
t2Prep["A"],
t2Prep["B"]
];

t2Blocks=t2Disc["Blocks"];

t2QuotientStates=Length[t2Blocks];

Print["QuotientStates=", t2QuotientStates];
Print["Splits=", t2Disc["Splits"]];

If[
t2QuotientStates != 3,
T2Stop[
"expected exactly 3 predictive quotient states on S3Q world"
]
];

t2TCCTFreezeHash=Hash[
ToString[
{
"S124-T2-STANDALONE",
t2WorldSeed,
t2CoreSeed,
t2Blocks,
t2Disc["Log"]
},
InputForm
],
"SHA256",
"HexString"
];

t2TCCTModel=<|
"Condition"->"Full",
"WorldSeed"->t2WorldSeed,
"CoreSeed"->t2CoreSeed,
"States"->t2Prep["States"],
"Blocks"->t2Blocks,
"TrainCounts"->t2Prep["AB"],
"StructuralStates"->Length[t2Prep["States"]],
"QuotientStates"->t2QuotientStates,
"Splits"->t2Disc["Splits"],
"FreezeHash"->t2TCCTFreezeHash

```

```

|>;

Print[""];
Print["TCCT FROZEN"];
Print["StructuralStates=",t2TCCTModel["StructuralStates"]];
Print["QuotientStates=",t2TCCTModel["QuotientStates"]];
Print["FreezeHash=",t2TCCTFreezeHash];

Put[
t2TCCTModel,
FileNameJoin[
{
t2OutputDirectory,
"S124_T2_TCCT_FROZEN_BEFORE_C.w1"
}
]
];

BlockRandom[
SeedRandom[t2PrototypeSeed];
t2SensorBases=
Normalize/@RandomVariate[
NormalDistribution[0.,1.],
{3,t2SensorDim}
]
];

T2Pos[pos_Integer]:=
Flatten[
Table[
{
Sin[pos/(10000.^(2.0*k/t2PosDim))],
Cos[pos/(10000.^(2.0*k/t2PosDim))]}
],
{k,0,t2PosDim/2-1}
]
];

T2Sensory[seq_List,seed_Integer]:=
BlockRandom[
SeedRandom[seed];
Module[{drift,gain},
drift=RandomReal[
{-t2SensorDrift,t2SensorDrift},
t2SensorDim
];
N[
Table[
gain=
1.+RandomReal[
{-t2SensorGainJitter,t2SensorGainJitter}
];
Join[
t2SensorScale*
gain*
t2SensorBases[[seq[[i]]]]+

```

```

drift+
RandomVariate[
NormalDistribution[
0.,
t2SensorNoise
],
t2SensorDim
],
T2Pos[i]
],
{i,1,Length[seq]}
]
]
];

t2InputDim=t2SensorDim+t2PosDim;

T2MakePerceptionRaw[
nPerClass_Integer,
seed_Integer
]:=
BlockRandom[
SeedRandom[seed];
RandomSample[
Flatten[
Table[
Table[
<|
"Input"->T2Sensory[
r3oSeqs[[j]],
seed+100000*j+i
],
"Label"->r3oSeqKeys[[j]]
|>,
{i,1,nPerClass}
],
{j,1,Length[r3oSeqs]}
],
1
]
]
];

T2ToRules[data_List]:=
(#["Input"]->#["Label"]&)/@data;

T2Block[
modelDim_Integer,
heads_Integer,
ffDim_Integer,
drop_?NumericQ
]:=
Module[{hd},
hd=Quotient[modelDim,heads];
NetGraph[

```

```

<|
"LN1"->NormalizationLayer[2,"Same"],
"Q"->NetMapOperator[
LinearLayer[{heads,hd}]
],
"K"->NetMapOperator[
LinearLayer[{heads,hd}]
],
"V"->NetMapOperator[
LinearLayer[{heads,hd}]
],
"Attention"->AttentionLayer[
"Dot",
"MultiHead"->True,
"Mask"->"Causal",
"ScoreRescaling"->"DimensionSqrt",
"Dropout"->drop
],
"Merge"->NetMapOperator[
NetChain[
{
FlattenLayer[],
LinearLayer[modelDim]
}
]
],
"ADrop"->DropoutLayer[drop],
"Res1"->ThreadingLayer[Plus],
"LN2"->NormalizationLayer[2,"Same"],
"FF"->NetMapOperator[
NetChain[
{
LinearLayer[ffDim],
ElementwiseLayer[Ramp],
DropoutLayer[drop],
LinearLayer[modelDim]
}
]
],
"FDrop"->DropoutLayer[drop],
"Res2"->ThreadingLayer[Plus]
|>,
{
NetPort["Input"]->"LN1",
"LN1"->"Q",
"LN1"->"K",
"LN1"->"V",
"Q"->NetPort["Attention","Query"],
"K"->NetPort["Attention","Key"],
"V"->NetPort["Attention","Value"],
"Attention"->"Merge",
"Merge"->"ADrop",
{
NetPort["Input"],
"ADrop"
}->"Res1",

```

```

"Res1"->"LN2",
"LN2"->"FF",
"FF"->"FDrop",
{
  "Res1",
  "FDrop"
}->"Res2"
},
"Input"->{"Varying",modelDim}
]
];

T2Transformer[]:=
Module[{blocks},
blocks=Table[
T2Block[
t2DModel,
t2Heads,
t2FF,
t2Dropout
],
{t2Layers}
];
NetChain[
Join[
{
NetMapOperator[
LinearLayer[t2DModel]
]
},
blocks,
{
NormalizationLayer[2,"Same"],
SequenceLastLayer[],
LinearLayer[
Length[r3oSeqKeys]
],
SoftmaxLayer[]
}
],
"Input"->{"Varying",t2InputDim},
"Output"->NetDecoder[
{"Class",r3oSeqKeys}
]
]
];

If[
Length[r3oSeqKeys]!=12,
T2Stop["Transformer output class count must be 12"]
];

Print[""];
Print["====="];
Print["PHASE 2: TRAIN TRANSFORMER PERCEPTION"];
Print["TCCT ALREADY FROZEN"];

```

```

Print["PROSPECTIVE C STILL DOES NOT EXIST"];
Print["====="];

t2PerceptionTrain=
T2MakePerceptionRaw[
t2PerceptionTrainPerClass,
t2PerceptionTrainSeed
];

t2PerceptionVal=
T2MakePerceptionRaw[
t2PerceptionValPerClass,
t2PerceptionValSeed
];

t2TrainRules=T2ToRules[t2PerceptionTrain];
t2ValRules=T2ToRules[t2PerceptionVal];

SeedRandom[t2PerceptionNetSeed];

t2Net0=NetInitialize[
T2Transformer[]
];

Print["TransformerInitialized=True"];
Print["OutputClasses=",Length[r3oSeqKeys]];
Print["Layers=",t2Layers];
Print["dModel=",t2DModel];
Print["Heads=",t2Heads];
Print["InputDim=",t2InputDim];

t2TrainingResult=
NetTrain[
t2Net0,
t2TrainRules,
All,
ValidationSet->t2ValRules,
MaxTrainingRounds->t2MaxRounds,
BatchSize->t2BatchSize,
LearningRate->t2LearningRate,
Method->"ADAM",
TrainingProgressMeasurements->{
"Accuracy",
"ErrorRate"
},
TrainingStoppingCriterion-><|
"Criterion"->"Loss",
"Patience"->t2Patience
|>,
TrainingProgressReporting->"Print",
TargetDevice->t2TargetDevice,
RandomSeeding->t2PerceptionNetSeed
];

t2FrozenPerception=
t2TrainingResult["TrainedNet"];

```

```

t2PerceptionValAccuracy=
NetMeasurements[
t2FrozenPerception,
t2ValRules,
"Accuracy",
BatchSize->t2BatchSize
];

Print[""];
Print["TRANSFORMER PERCEPTION FROZEN"];
Print["ValidationAccuracy=",t2PerceptionValAccuracy];

t2PerceptionModelFile=
FileNameJoin[
{
t2OutputDirectory,
"S124_T2_TRANSFORMER_FROZEN_BEFORE_C.wlnet"
}
];

Export[
t2PerceptionModelFile,
t2FrozenPerception
];

t2GlobalFreezeHash=
Hash[
ToString[
{
t2TCCTFreezeHash,
t2PerceptionNetSeed,
t2PerceptionValAccuracy,
t2SensorBases
},
InputForm
],
"SHA256",
"HexString"
];

Print[""];
Print["====="];
Print["GLOBAL FREEZE COMPLETE"];
Print["TCCTCoreChanged=False"];
Print["TCCTQFrozen=True"];
Print["TransformerFrozen=True"];
Print["ProspectiveCExists=False"];
Print["GlobalFreezeHash=",t2GlobalFreezeHash];
Print["====="];

Clear[
t2RawA,
t2RawB,
t2PerceptionTrain,
t2PerceptionVal,

```

```

t2TrainRules,
t2ValRules,
t2Net0,
t2TrainingResult
];

Print[""];
Print["====="];
Print["PHASE 3: PROSPECTIVE C-FIRST OPENING"];
Print["NO MODEL CHANGES AFTER THIS LINE"];
Print["====="];

t2RawC=Table[
R3OSim[
t2WorldSeed+63000000+i,
t2World
],
{i,1,t2BankC}
];

Print["ProspectiveExamples=",Length[t2RawC]];

t2OracleResult=
R3MEvaluate[
t2TCCTModel,
t2RawC,
t2FullData
];

T2SeqFromKey[key_String]:=
ToExpression[key];

t2TrueKeys=Lookup[
t2RawC,
"SeqKey"
];

t2SensoryC=Table[
T2Sensory[
T2SeqFromKey[
t2TrueKeys[[i]]
],
t2SensorCSeed+i
],
{i,1,Length[t2TrueKeys]}
];

Print["Running frozen Transformer perception..."];

t2PredictedKeys=Table[
If[
Mod[i,500]==0,
Print["Perception=",i,"/",Length[t2SensoryC]]
],
t2FrozenPerception[
t2SensoryC[[i]]

```



```

],
{i,1,Length[t2SensoryC]}
];

t2HistoryCorrect=
MapThread[
Boole[#1==#2]&,
{
t2PredictedKeys,
t2TrueKeys
}
];

t2PerceptionProspectiveAccuracy=
N[Mean[t2HistoryCorrect]];

Print["ProspectiveHistoryAccuracy=",t2PerceptionProspectiveAccuracy];

t2RawCHybrid=
MapThread[
Join[
KeyDrop[#1,{"SeqKey"}],
<|"SeqKey"->#2|>
]&,
{
t2RawC,
t2PredictedKeys
}
];

t2HybridResult=
R3MEvaluate[
t2TCCTModel,
t2RawCHybrid,
t2FullData
];

t2StateMap=t2FullData["SeqToState"];
t2BlockMap=R3MBlockMap[t2Blocks];

t2TrueQ=Table[
t2BlockMap[
t2StateMap[
t2TrueKeys[[i]]
]
],
{i,1,Length[t2TrueKeys]}
];

t2PredQ=Table[
t2BlockMap[
t2StateMap[
t2PredictedKeys[[i]]
]
],
{i,1,Length[t2PredictedKeys]}
];

```

```

];

t2QCorrect=
MapThread[
Boole[#1==#2]&,
{
t2PredQ,
t2TrueQ
}
];

t2QPreservation=
N[Mean[t2QCorrect]];

Print["QStatePreservation=",t2QPreservation];

BlockRandom[
SeedRandom[t2RandomControlSeed];
t2RandomKeys=
RandomChoice[
r3oSeqKeys,
Length[t2RawC]
]
];

t2RawCRandom=
MapThread[
Join[
KeyDrop[#1,{"SeqKey"}],
<|"SeqKey"->#2|>
]&,
{
t2RawC,
t2RandomKeys
}
];

t2RandomResult=
R3MEvaluate[
t2TCCTModel,
t2RawCRandom,
t2FullData
];

t2OracleBaseNLL=
t2OracleResult["BaseNLL"];

t2OracleQNLL=
t2OracleResult["QuotientNLL"];

t2HybridQNLL=
t2HybridResult["QuotientNLL"];

t2RandomQNLL=
t2RandomResult["QuotientNLL"];

```

```

t2OracleGain=
t2OracleBaseNLL-
t2OracleQNLL;

t2HybridGain=
t2OracleBaseNLL-
t2HybridQNLL;

t2RandomGain=
t2OracleBaseNLL-
t2RandomQNLL;

t2GainRetention=
If[
t2OracleGain>0.,
N[
t2HybridGain/
t2OracleGain
],
Indeterminate
];

t2HybridPenalty=
t2HybridQNLL-
t2OracleQNLL;

t2HybridBeatsRandom=
t2HybridQNLL<
t2RandomQNLL;

t2InterfacePass=
And[
t2PerceptionProspectiveAccuracy>=0.90,
t2QPreservation>=0.90,
t2HybridGain>0.,
NumericQ[t2GainRetention],
t2GainRetention>=0.85,
TrueQ[t2HybridBeatsRandom]
];

t2Diagnosis=
Which[
t2InterfacePass,
"TRANSFORMER_PERCEPTION_SUCCESSFULLY_DRIVES_FROZEN_TCCT_PREDICTIVE_QUOTIENT",
t2PerceptionProspectiveAccuracy<0.90,
"PERCEPTION_LAYER_NOT_ACCURATE_ENOUGH",
t2QPreservation<0.90,
"PERCEPTION_ERRORS_CHANGE_TOO_MANY_TCCT_Q_STATES",
t2HybridGain<=0.,
"HYBRID_LOSES_TCCT_PREDICTIVE_VALUE",
NumericQ[t2GainRetention]&&t2GainRetention<0.85,
"HYBRID_RETAINS_TOO_LITTLE_ORACLE_TCCT_GAIN",
!TrueQ[t2HybridBeatsRandom],
"HYBRID_NOT_BETTER_THAN_RANDOM_PERCEPTION",
True,
"PARTIAL_INTERFACE_RESULT"

```

```

];

Print[""];
Print["====="];
Print["S124-T2 FINAL SUMMARY"];
Print["====="];
Print["TCCTCoreChanged=False"];
Print["S3QHistories=", Length[r3oSeqs]];
Print["EventBagStates=", t2BagStates];
Print["Recent2States=", t2RecentStates];
Print["StructuralStates=", t2TCCTModel["StructuralStates"]];
Print["QuotientStates=", t2TCCTModel["QuotientStates"]];
Print["TCCTSplits=", t2TCCTModel["Splits"]];
Print["PerceptionValidationAccuracy=", t2PerceptionValAccuracy];
Print["PerceptionProspectiveAccuracy=", t2PerceptionProspectiveAccuracy];
Print["QStatePreservation=", t2QPreservation];
Print["OracleBaseNLL=", t2OracleBaseNLL];
Print["OracleTCCT_QNLL=", t2OracleQNLL];
Print["HybridTransformerTCCT_QNLL=", t2HybridQNLL];
Print["RandomPerception_QNLL=", t2RandomQNLL];
Print["OracleTCCTGain=", t2OracleGain];
Print["HybridTCCTGain=", t2HybridGain];
Print["RandomControlGain=", t2RandomGain];
Print["HybridGainRetention=", t2GainRetention];
Print["HybridPenaltyVsOracle=", t2HybridPenalty];
Print["HybridBeatsRandom=", t2HybridBeatsRandom];
Print["STRICT INTERFACE PASS=", t2InterfacePass];
Print["DIAGNOSIS=", t2Diagnosis];
Print["====="];

t2Summary=<|
"Stage"->"S124-T2-STANDALONE",
"Purpose"->"TransformerPerceptionToFrozenTCCTInterfaceGate",
"TCCTCoreChanged"->False,
"S3QHistories"->Length[r3oSeqs],
"EventBagStates"->t2BagStates,
"Recent2States"->t2RecentStates,
"WorldSeed"->t2WorldSeed,
"CoreSeed"->t2CoreSeed,
"StructuralStates"->t2TCCTModel["StructuralStates"],
"QuotientStates"->t2TCCTModel["QuotientStates"],
"TCCTSplits"->t2TCCTModel["Splits"],
"PerceptionValidationAccuracy"->t2PerceptionValAccuracy,
"PerceptionProspectiveAccuracy"->t2PerceptionProspectiveAccuracy,
"QStatePreservation"->t2QPreservation,
"OracleBaseNLL"->t2OracleBaseNLL,
"OracleTCCTQNLL"->t2OracleQNLL,
"HybridTransformerTCCTQNLL"->t2HybridQNLL,
"RandomPerceptionQNLL"->t2RandomQNLL,
"OracleGain"->t2OracleGain,
"HybridGain"->t2HybridGain,
"RandomGain"->t2RandomGain,
"HybridGainRetention"->t2GainRetention,
"HybridPenaltyVsOracle"->t2HybridPenalty,
"HybridBeatsRandom"->t2HybridBeatsRandom,
"StrictInterfacePass"->t2InterfacePass,

```

```
"Diagnosis"->t2Diagnosis,  
"TCCTFreezeHash"->t2TCCTFreezeHash,  
"GlobalFreezeHash"->t2GlobalFreezeHash,  
"ProspectiveGeneratedAfterBothFrozen"->True  
|>;  
  
t2SummaryFile=  
FileNameJoin[  
{  
t2OutputDirectory,  
"S124_T2_summary.w1"  
}  
];  
  
Put[  
t2Summary,  
t2SummaryFile  
];  
  
Print["SummaryFile=",t2SummaryFile];  
Print[""];  
Print["====="];  
Print["S124-T2 COMPLETE"];  
Print["====="];
```

=====

S124-T2 STANDALONE

TRANSFORMER PERCEPTION -> FROZEN TCCT-Q INTERFACE GATE

=====

WolframVersion=15.0.0 for Microsoft Windows (64-bit) (May 26, 2026)

Date=Wed 19 Aug 2026 21:32:54

=====

S3Q HISTORY PRECHECK

=====

HistoryCount=12

AllEventBags={2,2,2}=True

AllRecent2={1,2}=True

CoreDepth=1 Histories=3

CoreDepth=2 Histories=9

CoreDepth=3 Histories=27

CoreDepth=4 Histories=81

CoreDepth=5 Histories=243

CoreDepth=6 Histories=729

RestrictedFullStructuralStates=12

EventBagStates=1

Recent2States=1

TCCT CORE PRECHECK=PASS

=====

PHASE 1: DISCOVERY A + VALIDATION B

PROSPECTIVE C DOES NOT EXIST

=====

StructuralStates=12

Discovering predictive quotient...

ACCEPT SPLIT=1 STATES=12 PROBE=1 GAIN=0.382862

QuotientStates=3

Splits=1

TCCT FROZEN

StructuralStates=12

QuotientStates=3

FreezeHash=1c26c77fcdd1cb1d1c329612f786f31e759e27d983a5e12f457485478061e6d0

=====

PHASE 2: TRAIN TRANSFORMER PERCEPTION

TCCT ALREADY FROZEN

PROSPECTIVE C STILL DOES NOT EXIST

=====

TransformerInitialized=True

OutputClasses=12

Layers=2

dModel=64

Heads=4

InputDim=20

Starting training.

Optimization Method: ADAM

Beta1: 9.00×10^{-1}

Beta2: 9.99×10^{-1}

Epsilon: 1.00×10^{-5}

Gradient Clipping: --
 L2 Regularization: --
 Learning Rate: 3.00×10^{-4}
 Learning Rate Schedule: --
 Weight Clipping: --

Device: CPU

Batch Size: 64

Batches Per Round: 57

Training Examples: 3600

%	round	batch	examples	inputs	learning	time	time	\
>	current	round	test	current	round	test	current\	
>	round	test						
	/20	/57	processed	/second	rate	elapsed	left	\
>	loss	loss	loss	acc.	acc.	acc.	error	\
>	error	error						
5	1	57	3648	2045	7.08×10^{-5}	2s	45s	\
>	2.42×10^0	2.42×10^0	1.78×10^0	16.3%	16.3%	52.0%		\
>	83.7%	83.7%	48.0%					
10	2	57	7296	1994	9.85×10^{-5}	4s	40s	\
>	1.36×10^0	1.36×10^0	6.24×10^{-1}	54.2%	54.2%	90.58%		\
>	45.8%	45.8%	9.42%					
15	3	57	10944	1920	1.19×10^{-4}	7s	37s	\
>	6.18×10^{-1}	6.18×10^{-1}	1.95×10^{-1}	86.7%	86.7%	99.833%		\
>	13.3%	13.3%	0.167%					
20	4	57	14592	2009	1.35×10^{-4}	9s	35s	\
>	2.97×10^{-1}	2.97×10^{-1}	7.50×10^{-2}	96.66%	96.66%	100%		\
>	3.34%	3.34%	0%					
25	5	57	18240	2041	1.49×10^{-4}	11s	32s	\
>	1.65×10^{-1}	1.65×10^{-1}	3.80×10^{-2}	98.68%	98.68%	100%		\
>	1.32%	1.32%	0%					
30	6	57	21888	2066	1.61×10^{-4}	13s	30s	\
>	1.12×10^{-1}	1.12×10^{-1}	2.24×10^{-2}	99.068%	99.068%	100%		\
>	0.932%	0.932%	0%					
35	7	57	25536	2073	1.72×10^{-4}	15s	27s	\
>	7.60×10^{-2}	7.60×10^{-2}	1.55×10^{-2}	99.534%	99.534%	100%		\
>	0.466%	0.466%	0%					
40	8	57	29184	2079	1.82×10^{-4}	17s	25s	\

>	5.79* ⁻²	5.79* ⁻²	1.11* ⁻²	99.616%	99.616%	100%	\
>	0.384%	0.384%	0%				
45	9	57	32832	2055	1.90* ⁻⁴	19s	23s \
>	4.40* ⁻²	4.40* ⁻²	8.51* ⁻³	99.9178%	99.9178%	100%	\
>	0.0822%	0.0822%	0%				
47	10	25	34432	1507	1.94* ⁻⁴	21s	23s \
>	3.71* ⁻²	4.40* ⁻²	8.51* ⁻³	99.875%	99.9178%	100%	\
>	0.125%	0.0822%	0%				
52	11	25	38080	1977	2.01* ⁻⁴	23s	21s \
>	3.45* ⁻²	3.54* ⁻²	6.80* ⁻³	99.808%	99.863%	100%	\
>	0.192%	0.137%	0%				
57	12	25	41728	1966	2.08* ⁻⁴	25s	19s \
>	3.01* ⁻²	3.32* ⁻²	5.40* ⁻³	99.726%	99.698%	100%	\
>	0.274%	0.302%	0%				
62	13	24	45312	1908	2.14* ⁻⁴	27s	16s \
>	2.53* ⁻²	2.71* ⁻²	4.54* ⁻³	99.9163%	99.890%	100%	\
>	0.0837%	0.110%	0%				
67	14	24	48960	1941	2.19* ⁻⁴	29s	14s \
>	2.22* ⁻²	2.30* ⁻²	3.65* ⁻³	99.808%	99.863%	100%	\
>	0.192%	0.137%	0%				
72	15	22	52480	1877	2.24* ⁻⁴	31s	12s \
>	1.92* ⁻²	2.15* ⁻²	3.20* ⁻³	99.9148%	99.808%	100%	\
>	0.0852%	0.192%	0%				
77	16	21	56064	1877	2.29* ⁻⁴	33s	10s \
>	2.05* ⁻²	1.79* ⁻²	2.69* ⁻³	99.749%	99.863%	100%	\
>	0.251%	0.137%	0%				
82	17	20	59648	1840	2.34* ⁻⁴	35s	8s \
>	1.65* ⁻²	1.97* ⁻²	2.45* ⁻³	99.888%	99.781%	100%	\
>	0.112%	0.219%	0%				
87	18	18	63168	1795	2.38* ⁻⁴	37s	6s \
>	1.42* ⁻²	1.44* ⁻²	2.16* ⁻³	99.858%	99.890%	100%	\
>	0.142%	0.110%	0%				
91	19	17	66752	1747	2.41* ⁻⁴	39s	4s \
>	1.32* ⁻²	1.29* ⁻²	1.85* ⁻³	99.9442%	99.9452%	100%	\


```

> 0.0558% 0.0548% 0%
96 20 16 70336 1771 2.45*^-4 41s 2s \

> 1.27*^-2 1.33*^-2 1.72*^-3 99.888% 99.890% 100% \

> 0.112% 0.110% 0%

```

TRANSFORMER PERCEPTION FROZEN
ValidationAccuracy=1.

```

=====
GLOBAL FREEZE COMPLETE
TCCTCoreChanged=False
TCCTQFrozen=True
TransformerFrozen=True
ProspectiveCExists=False
GlobalFreezeHash=2c57e14cdd02ff7136a5fe9de75696d6cc787ccbd492dd629a62e4613834\

```

```

> 2461
=====

```

```

=====
PHASE 3: PROSPECTIVE C-FIRST OPENING
NO MODEL CHANGES AFTER THIS LINE
=====

```

```

ProspectiveExamples=6000
Running frozen Transformer perception...
Perception=500/6000
Perception=1000/6000
Perception=1500/6000
Perception=2000/6000
Perception=2500/6000
Perception=3000/6000
Perception=3500/6000
Perception=4000/6000
Perception=4500/6000
Perception=5000/6000
Perception=5500/6000
Perception=6000/6000
ProspectiveHistoryAccuracy=1.
QStatePreservation=1.

```

```

=====
S124-T2 FINAL SUMMARY
=====

```

```

TCCTCoreChanged=False
S3QHistories=12
EventBagStates=1
Recent2States=1
StructuralStates=12
QuotientStates=3
TCCTSplits=1
PerceptionValidationAccuracy=1.
PerceptionProspectiveAccuracy=1.
QStatePreservation=1.

```

```
OracleBaseNLL=1.19004
OracleTCCT_QNLL=0.817542
HybridTransformerTCCT_QNLL=0.817542
RandomPerception_QNLL=1.6651
OracleTCCTGain=0.372503
HybridTCCTGain=0.372503
RandomControlGain=-0.475056
HybridGainRetention=1.
HybridPenaltyVsOracle=0.
HybridBeatsRandom=True
STRICT INTERFACE PASS=True
DIAGNOSIS=TRANSFORMER_PERCEPTION_SUCCESSFULLY_DRIVES_FROZEN_TCCT_PREDICTIVE_Q\
```

```
> UOTIENT
```

```
=====
SummaryFile=E:\TCCT_CODEX_HANDOFF_2026-08-13\S97A_ReadoutBaseline_Development\
```

```
> \S124_T2_Output\S124_T2_summary.wl
```

```
=====
S124-T2 COMPLETE
=====
```

In []: